EVENTFORCE EVENT MANAGEMENT SYSTEM – SALESFORCE IMPLEMENTATION

Naan Mudhalvan – Salesforce Developer Project Report

College: A.V.C College of Engineering | Team Members: S.Abinaya(TL), K.Abinesha, K.Aishwarya, V.Dharanika, M.Gopika.

User Story

EventForce is a Salesforce-based application built for an event-management company that handles weddings, corporate programs, concerts and similar functions. Today the company tracks clients, bookings, halls and service providers in separate notebooks and spreadsheets, so dates clash, reminders are missed and nobody has a clear picture of what is happening next week. EventForce brings all of this into one Salesforce org. Event staff record clients, events, venues, vendors and feedback as connected records. Flows send reminders automatically, an Approval Process controls cancellations, and Apex code stops two events from taking the same venue on the same date. A Lightning App with Reports and Dashboards gives managers a live view of upcoming events, budgets and vendor activity.

1. INTRODUCTION

1.1 Project Overview

EventForce is a CRM solution developed on the Salesforce platform to organise the complete event life cycle: enquiry, booking, venue allocation, vendor assignment, reminders, cancellation, completion and feedback.

The solution is built with six custom objects (Event, Client, Venue, Vendor, Event Vendor and Feedback), validation rules, a formula field, a record-triggered Flow, an Approval Process, Apex triggers and handler classes, Batch and Scheduled Apex, role-based security, a Lightning App, Reports and a Dashboard.

The project was implemented in a Salesforce Developer Edition org for academic purposes, following the Naan Mudhalvan project guidelines.

1.2 Purpose

The purpose of the project is to replace scattered manual tracking with one automated platform. Specifically, the system is designed to:

- Store client, event, venue, vendor and feedback data in one place.
- Make booking a venue for an event a simple, guided task.
- Block double bookings automatically using Apex.
- Send clients a reminder email before their event without manual effort.
- Route every cancellation through a manager approval.
- Give managers live reports and a dashboard for decisions.
- Restrict data access so each user sees only what their job needs.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Event Operations Manager	see all upcoming events at a glance	details sit in different files and notebooks	there is no single system for the whole team	confused and under pressure
PS-2	Event Coordinator	book an event for a client quickly	I must re-type client and venue details in many places	records are not linked to each other	tired and error-prone
PS-3	Venue Manager	keep the hall calendar correct	two events can be given the same hall on the same day	availability is checked by memory	anxious and embarrassed
PS-4	Vendor Manager	allot caterers, decorators and photographers	one vendor may be needed for many events	there is no record of who is assigned where	stretched and uncertain
PS-5	Client Care Executive	remind clients before their function	reminders are sent by phone one by one	nobody owns the follow-up	stressed and forgetful
PS-6	Event Administrator	approve or reject cancellations	requests arrive by call or message	there is no approval trail	uneasy about accountability
PS-7	Event Coordinator	collect feedback after an event	ratings are on paper or WhatsApp	they are not linked to the client or event	worried about service quality
PS-8	Management Team	review performance every month	reports must be prepared manually	data is not organised for analysis	unable to plan confidently

2.2 Empathy Map Canvas

User Persona: Event Operations Manager

Primary responsibility: Make sure every event is booked correctly, staffed with the right vendors, held at a free venue and closed with client feedback.

Quadrant	Details
Think & Feel	Wants every event to run smoothly; fears last-minute clashes and unhappy clients.
Hear	Clients asking for updates, vendors asking for confirmation, management asking for numbers.
See	Many spreadsheets, phone messages, hand-written booking registers, calendars that do not match.
Say & Do	Calls vendors, checks hall dates, updates sheets, chases cancellations and reminders.
Pain	Double bookings, missed reminders, no approval record, hours spent preparing reports.
Gain	One system for all records, automatic alerts, clear approvals and ready-made dashboards.

2.3 Brainstorming & Idea Prioritization

Ideas considered

Idea	How it would work	Main drawback
Paper registers	Handwritten booking and vendor books	Slow, easy to lose, no search
Excel sheets	Shared spreadsheets for bookings	No automation, version conflicts
Custom web application	Build a new website and database	High cost and long development time
Ready-made event SaaS	Subscribe to a packaged tool	Limited customisation, recurring fees
Salesforce CRM	Configure objects, flows and Apex on the platform	Needs skilled configuration

Prioritization (score 1 = poor, 5 = excellent)

Idea	Cost	Automation	Scalability	Ease of maintenance	Live visibility	Total
Paper registers	5	1	1	2	1	10
Excel sheets	5	2	2	3	2	14
Custom web application	2	4	4	1	3	14
Ready-made event SaaS	2	3	3	4	3	15
Salesforce CRM	4	5	5	4	5	**23**

Decision: Salesforce CRM scored highest because it offers point-and-click automation, built-in security, reports and dashboards, and room to grow without rebuilding the system.

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stage	Journey stage	What the customer / team does	What EventForce does
1	Enquiry	Client contacts the company with an idea and budget	Client record is created with contact details
2	Event creation	Coordinator enters type, date and guest count	Event record is saved; budget estimate is calculated by formula
3	Venue selection	Coordinator picks a hall	Apex checks the date; double booking is blocked; venue becomes Reserved on confirmation
4	Vendor allocation	Vendor Manager assigns caterer, decor, photography	Event Vendor junction records link vendors to the event
5	Confirmation and reminder	Event status is set to Confirmed	Flow emails the client two days before the event
6	Event day	Team executes the event	Dashboard shows today's and upcoming events
7	Change or cancellation	Client asks to cancel	Status moves to Cancellation Requested; Approval Process runs; venue is released if approved
8	Closure and feedback	Event ends, client rates the service	Nightly batch marks the event Completed; Feedback record is stored

3.2 Solution Requirements

Functional requirements

FR No	Requirement	Description
FR-1	Event management	Create and track events with type, date, status, guest count and budget
FR-2	Client management	Store client contact and location details with email validation
FR-3	Venue management	Maintain venue address, map link, capacity and availability
FR-4	Vendor management	Maintain vendor services, contact details and status
FR-5	Event–vendor assignment	Many-to-many link between events and vendors through a junction object
FR-6	Double-booking prevention	Block a second event at the same venue on the same date
FR-7	Venue status update	Mark venue Reserved or Open automatically from event status
FR-8	Automated reminder	Email the client two days before a confirmed event
FR-9	Cancellation approval	Approval Process with email alerts and status updates
FR-10	Feedback management	Capture rating and comments linked to client and event
FR-11	Event completion automation	Nightly batch job closes past events
FR-12	Reports and dashboards	Upcoming events, budget and vendor insights
FR-13	Data migration	Load initial records using Data Import Wizard

Non-functional requirements

NFR	Description
Usability	Lightning App with simple tabs and guided record pages
Security	Profiles, roles, permission sets, OWD and sharing rules
Reliability	Validation rules, approvals and Apex checks prevent wrong data
Performance	Bulk-safe Apex that avoids queries inside loops
Availability	Cloud access from any browser for authorised users
Scalability	Can grow in users, records and features without redesign
Maintainability	Declarative automation and clearly named Apex classes
Data integrity	Validated imports and required fields keep records consistent

3.3 Data Flow Diagram

Main flow

```
Client / Event Team  |  vEnter client and event details  |  vSalesforce EventForce (validation rules run)  |  vEvent, Client, Venue, Vendor records stored  |  vApex checks venue and date  -->  conflict? error shown  |  vVendors assigned (Event Vendor records)  |  vStatus = Confirmed  -->  Venue = Reserved  |  vFlow sends reminder 2 days before event  |  vEvent held  -->  Nightly batch sets Completed  |  vFeedback captured  -->  Reports and Dashboard
```

Supporting flows

- **Cancellation:** Coordinator sets status to Cancellation Requested → Approval Process → Manager approves or rejects → status becomes Cancelled or Cancellation Rejected → email alerts sent.
- **Venue check:** Event saved → Apex trigger → query same venue and date (ignoring cancelled events) → block or allow.
- **Completion:** Event date passes → Scheduled Apex runs at night → Batch Apex updates Confirmed events to Completed.

3.4 Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition
User interface	Lightning Experience, Lightning App
Data model	Custom objects, lookup and master-detail relationships
Declarative logic	Formula fields, validation rules, record-triggered Flow
Approval	Salesforce Approval Process with email templates
Programming	Apex classes, triggers, Batch Apex, Schedulable Apex
Security	Profiles, roles, permission sets, OWD, sharing rules
Data loading	Data Import Wizard
Analytics	Reports and Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Problem	EventForce solution	Benefit
Data scattered across files	Six linked custom objects	One source of truth
Manual, repeated data entry	Lookups to Client and Venue, formula budget	Faster and more accurate booking
Venue clashes	Apex trigger with date check	No double bookings
Vendor tracking	Event Vendor junction object	Clear vendor-to-event mapping
Missed reminders	Scheduled path in a record-triggered Flow	Reminders sent automatically
Uncontrolled cancellations	Event_Cancellation_Approval process	Manager decision recorded
Feedback not linked	Feedback with Client and Event lookups and a lookup filter	Clean feedback history
No overview	Reports and Operations Dashboard	Quick decisions
Everyone sees everything	Profiles, roles, OWD and sharing rules	Access by need

4.2 Proposed Solution

Parameter	Description
Problem	Event operations are manual, scattered and error-prone
Solution	A Salesforce Lightning application with automation and Apex logic
Innovation	Venue conflict blocking, automatic reminders, approval-based cancellation, nightly completion job
Impact	Fewer clashes, faster coordination, better client communication
Business model	Internal CRM for an event-management company
Scalability	New objects, flows and users can be added without rebuilding

4.3 Solution Architecture

Layer	Components
Data	Event__c, Client__c, Venue__c, Vendor__c, Event_Vendor__c, Feedback__c
Declarative automation	Formula field, validation rules, record-triggered Flow, Approval Process, email templates
Apex	VenueAvailabilityHandler, EventVenueSyncTrigger, BlockDoubleBooking, BatchMarkEventsCompleted, EventCompletionScheduler
Security	Profiles, roles, permission set, OWD, sharing rule
Presentation	Custom tabs, EventForce Lightning App
Analytics	Report folder, Upcoming Events by Month report, EventForce Operations Dashboard
Data load	Data Import Wizard

Relationships

Relationship	Type
Event → Client	Lookup
Event → Venue	Lookup
Feedback → Event	Lookup (with filter)
Feedback → Client	Lookup
Event Vendor → Event	Master-Detail
Event Vendor → Vendor	Master-Detail

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The team followed the Agile approach: requirements were grouped into epics, split into user stories, sized with story points and delivered in four six-day sprints. Velocity from completed sprints was used to plan the next one.

Product backlog and sprint schedule

Sprint	Epic	Story no.	User story / task	Points	Priority	Member
Sprint-1	Event Management	US-01	As an Event Coordinator, I want to create events with type, date, guest count and status so that all events are recorded in one format.	5	High	Member 1
Sprint-1	Client & Venue Management	US-02	As an Event Coordinator, I want to link a client and a venue to each event so that all details are available together.	5	High	Member 2
Sprint-1	Data Quality	US-03	As an Administrator, I want validation rules so that wrong emails, past dates and over-capacity guest counts are rejected.	3	Medium	Member 3
Sprint-2	Vendor Management	US-04	As a Vendor Manager, I want to assign several vendors to an event so that every service is covered.	5	High	Member 4
Sprint-2	Notifications	US-05	As a Client, I want an email two days before my event so that I can prepare.	5	High	Member 1
Sprint-2	Feedback	US-06	As a Coordinator, I want to record client ratings	3	Medium	Member 2

			and comments against the event so that service quality can be reviewed.			
Sprint-3	Approval Management	US-07	As an Administrator, I want cancellations to need manager approval so that they are controlled.	5	High	Member 3
Sprint-3	Venue Logic (Apex)	US-08	As a Venue Manager, I want double bookings blocked and venue status updated automatically so that the calendar stays correct.	8	High	Member 4
Sprint-4	Security	US-09	As an Administrator, I want role-based access so that users see only what they need.	5	High	Member 1
Sprint-4	Reports & Dashboards	US-10	As Management, I want reports and a dashboard so that I can monitor operations.	5	Medium	Member 2
Sprint-4	Data Migration & Testing	US-11	As a Team, we want to import sample data and test every feature so that the system is ready for demo.	3	Medium	Member 3

Project tracker

Sprint	Total story points	Duration	Start date	End date (planned)	Points completed	Release date (actual)
Sprint-1	13	6 days	03 Aug 2026	08 Aug 2026	13	08 Aug 2026
Sprint-2	13	6 days	10 Aug 2026	15 Aug 2026	13	15 Aug 2026
Sprint-3	13	6 days	17 Aug 2026	22 Aug 2026	13	22 Aug 2026
Sprint-4	13	6 days	24 Aug 2026	29 Aug 2026	13	29 Aug 2026

Velocity

Total story points = 13 + 13 + 13 + 13 = 52. Average velocity = 52 ÷ 4 sprints = **13 story points per sprint**, which is about 2.17 points per day (13 ÷ 6 days).

Burndown (story points remaining)

Point in time	Ideal remaining	Actual remaining
Project start	52	52
End of Sprint-1	39	39
End of Sprint-2	26	26
End of Sprint-3	13	13
End of Sprint-4	0	0

[Insert burndown chart screenshot here]

6. PROJECT DEVELOPMENT PHASE

Phase 1: Requirement Analysis & Planning

The goal of this phase was to define what EventForce must do before any configuration began: manage clients, bookings, venues, vendors and feedback in one Salesforce org, automate reminders and cancellations, and give management live reports.

Functional scope

- Six custom objects: Event, Client, Venue, Vendor, Event Vendor (junction) and Feedback.
- Relationships: lookup, master-detail and many-to-many (Event ↔ Vendor).
- Validation rules and a formula field for clean data and budget estimation.
- A record-triggered Flow for client reminders.
- An Approval Process for event cancellation.
- Apex for venue status updates, double-booking prevention and nightly event completion.
- Profiles, roles, permission set, OWD and sharing rules.
- Lightning App, report and dashboard.

Stakeholder mapping

Stakeholder	Responsibility in EventForce
Client	Requests events, receives reminders, gives feedback
Event Coordinator	Creates events, selects venues, raises cancellation requests
Vendor Manager	Maintains vendors and assigns them to events
Venue Manager	Keeps venue details and availability correct
Event Administrator	Approves cancellations, controls security and data

Execution roadmap

1. Create the Developer org and the custom objects.
2. Create tabs, fields and relationships.
3. Add validation rules.
4. Create profiles, roles, users, permission set and sharing settings.
5. Build email templates and the Approval Process.
6. Build the reminder Flow.
7. Write Apex classes, triggers, batch and scheduler.
8. Build the Lightning App, report and dashboard.
9. Import sample data and test.

Phase 2: Backend Development & Configuration

This phase built the working core of EventForce. Declarative tools (validation rules, formula field, Flow, Approval Process) handle routine automation, while Apex handles logic that clicks alone cannot do, such as comparing a new event against existing bookings. Security configuration makes sure each user sees only the records relevant to their job.

Milestone 1: Salesforce Developer Account

Salesforce is a cloud CRM platform used to manage customers, sales, service and business processes. Developer Edition is a free org meant for learning and building projects.

Activity 1: Create the developer org

1. Open the Salesforce Developer Edition sign-up page (developer.salesforce.com/signup).
2. Fill in first name, last name, email, role (Developer), company (college name), country (India) and postal code.
3. Give a unique username in email format, for example *yourname@eventforce.dev*.
4. Click **Sign me up**.

Activity 2: Activate the account

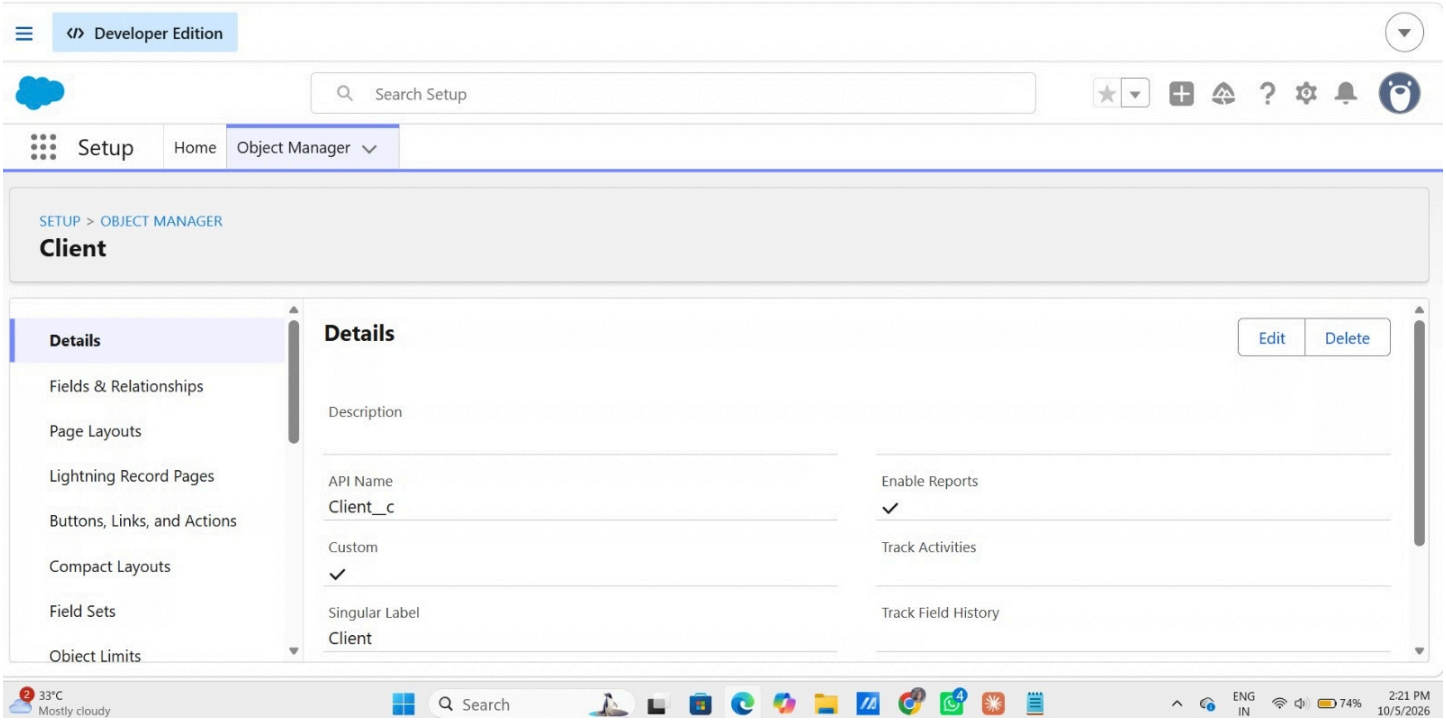
1. Open the verification email (it can take 5–10 minutes) and click **Verify Account**.
2. Set a password, choose a security question and click **Change Password**.
3. You are taken to the Salesforce Setup home page.

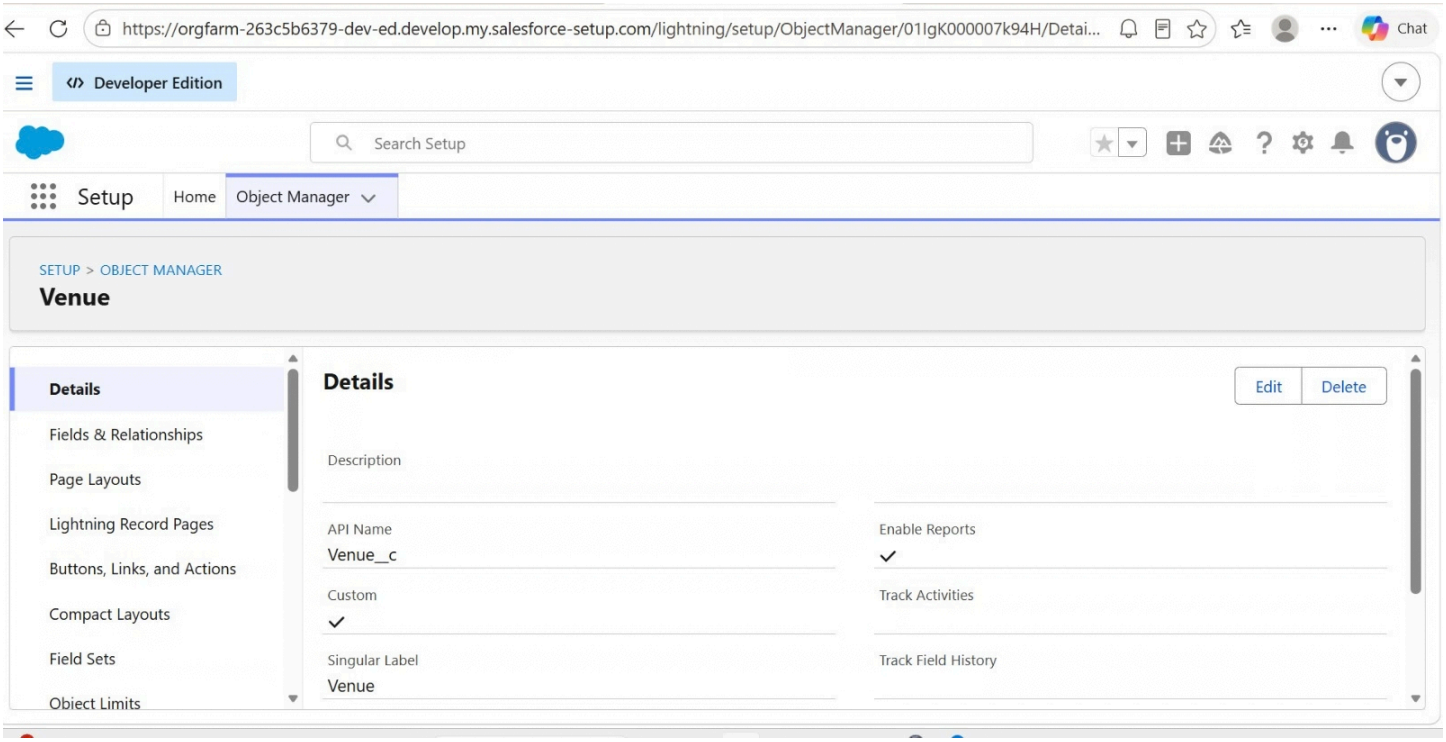
Data architecture created in this project

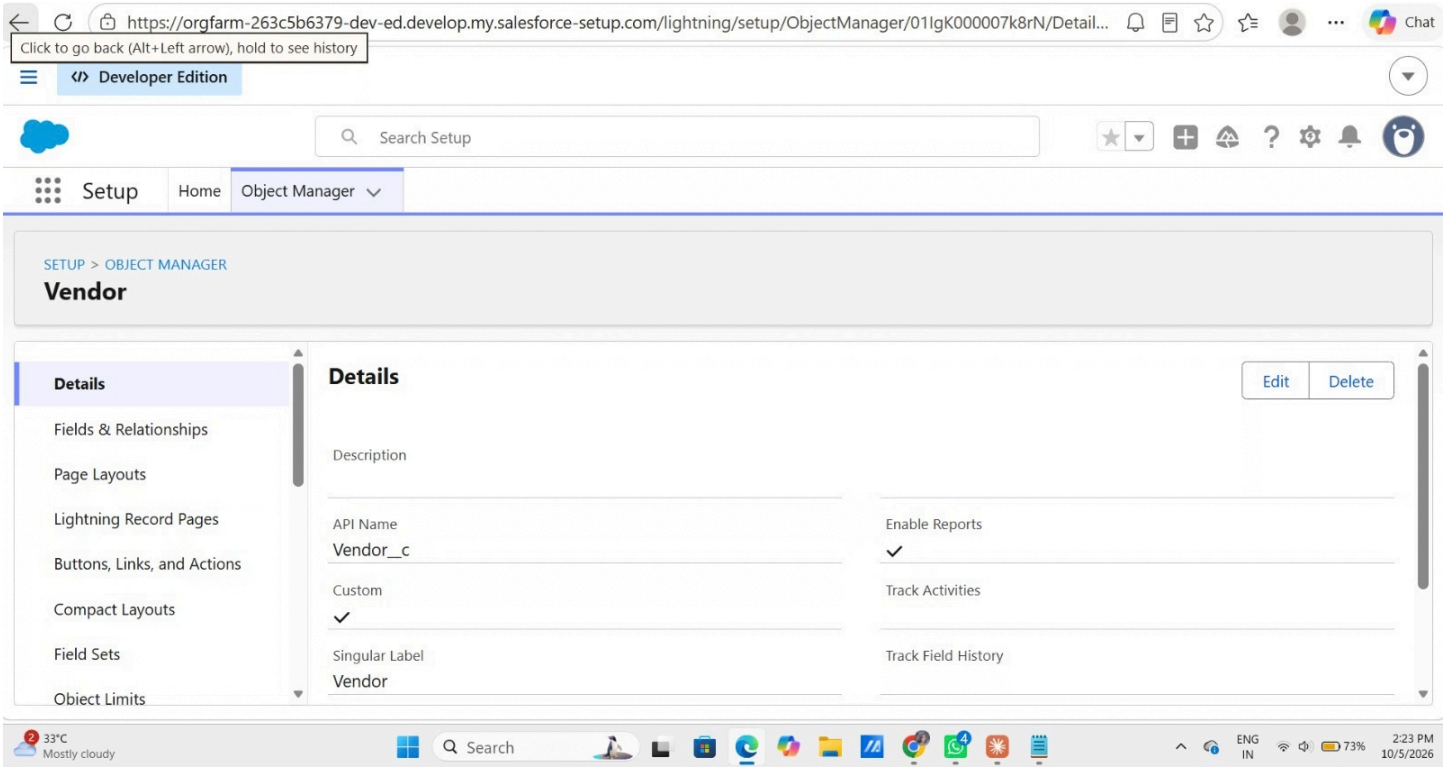
Object	API name	Purpose	Key fields
Event	Event__c	Each booking	Event Type, Event Date, Guest Count, Event Status, Estimated Budget, Client, Venue
Client	Client__c	Person or company booking events	Email, Phone, Address, Country, City
Venue	Venue__c	Halls and locations	Address, Map Location, Seating Capacity, Availability Status
Vendor	Vendor__c	Service providers	Service Type, Email, Phone, Status
Event Vendor	Event_Vendor__c	Junction of Event and Vendor	Event, Vendor, Agreed Fee, Service Notes
Feedback	Feedback__c	Post-event rating	Rating, Comments, Event, Client

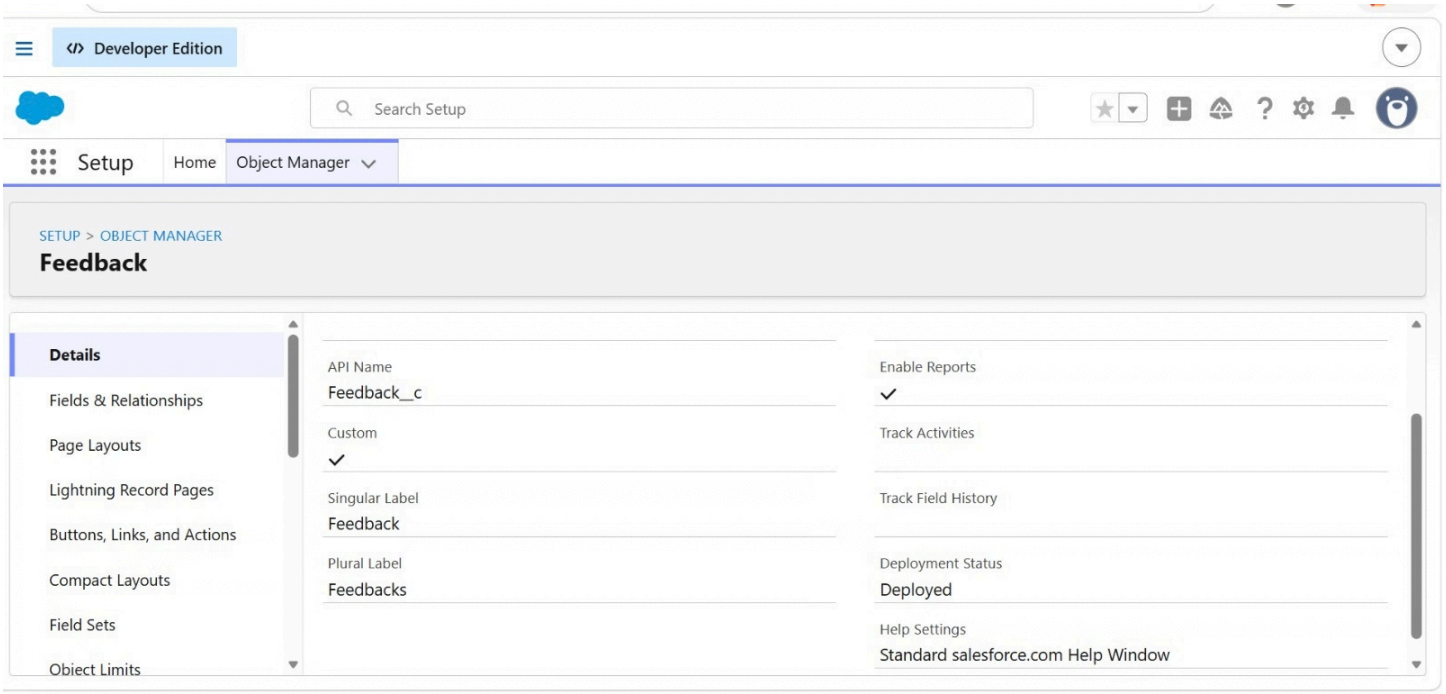
Milestone 2: Objects

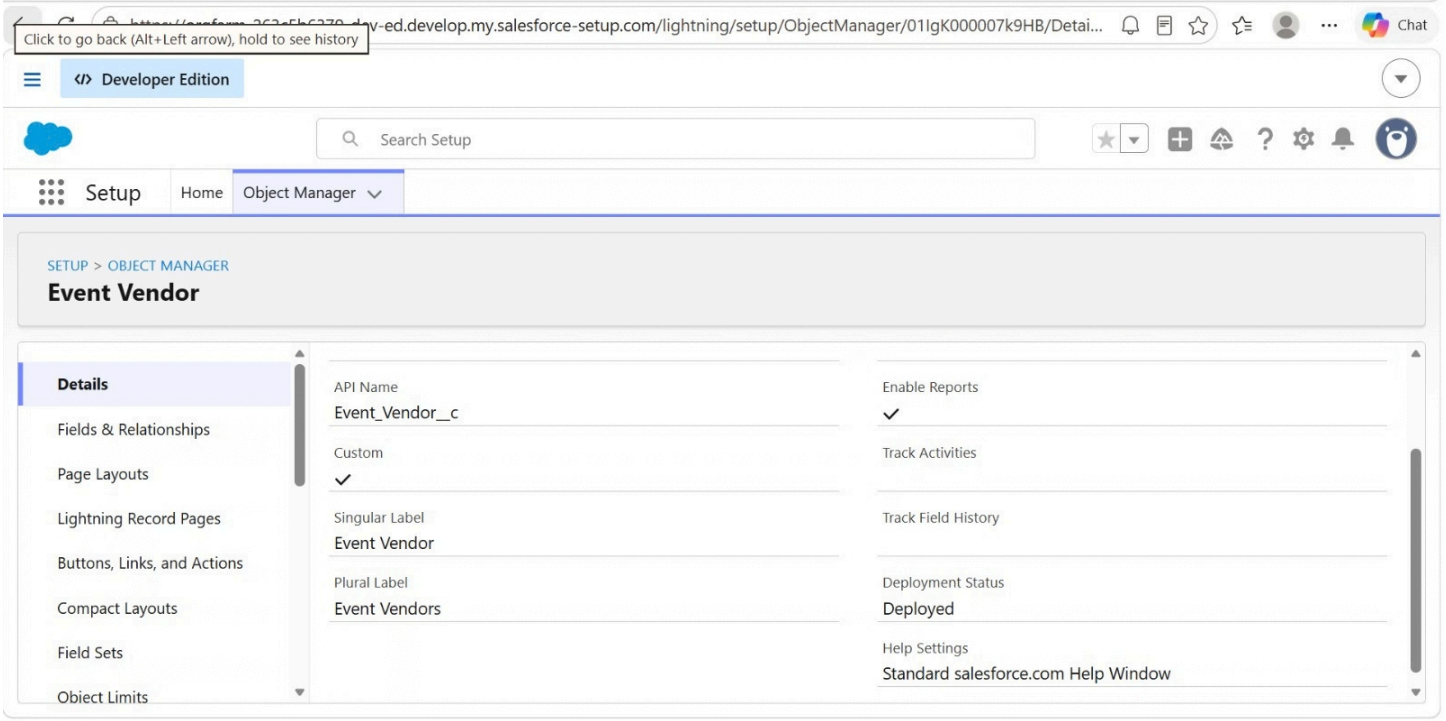
An object in Salesforce is like a database table: fields are the columns and records are the rows. EventForce uses five main custom objects plus one junction object.

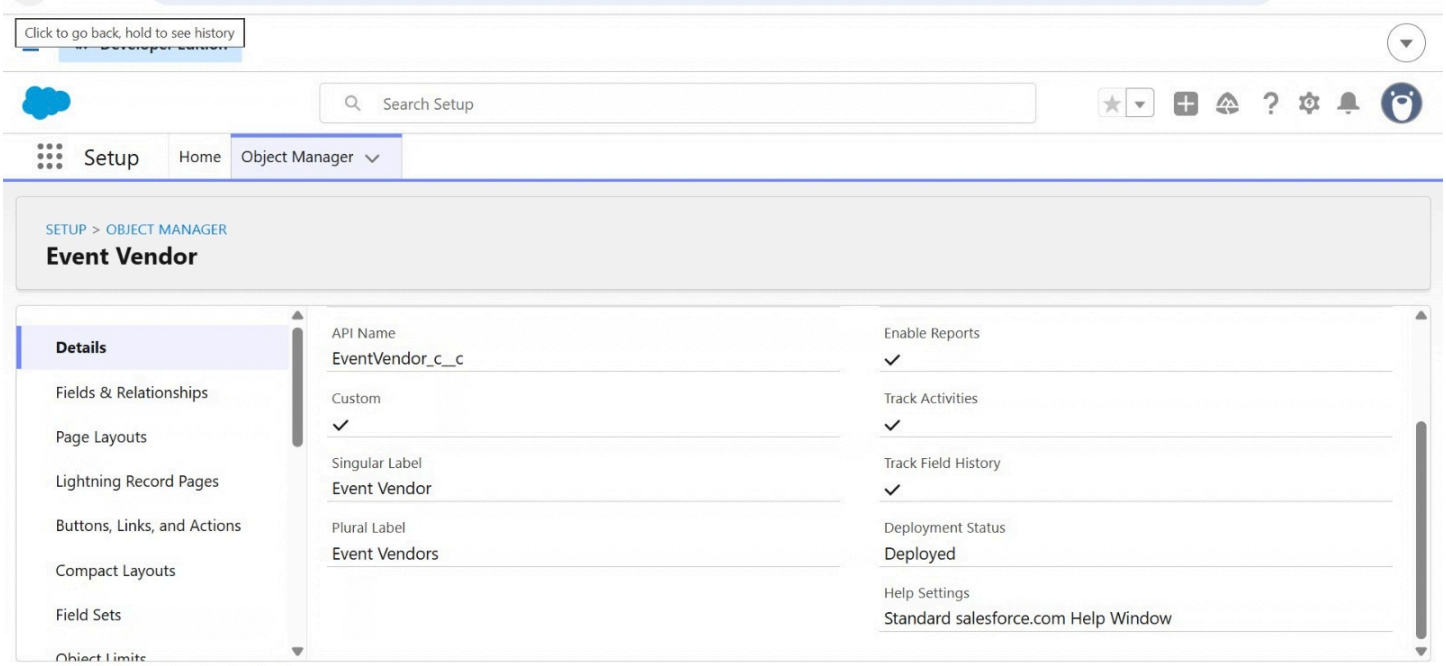



https://orgfarm-263c5b6379-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01IgK000007k94H/Detai...
Chat
Developer Edition
Search Setup
Setup
Home
Object Manager
SETUP > OBJECT MANAGER
Venue
Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Details
Edit
Delete
Description
API Name
Venue__c
Enable Reports
Custom
Track Activities
Singular Label
Venue
Track Field History


https://orgfarm-263c5b6379-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01IgK000007k8rN/Detail...
Click to go back (Alt+Left arrow), hold to see history
Chat
Developer Edition
Search Setup
Setup
Home
Object Manager
SETUP > OBJECT MANAGER
Vendor
Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Details
Edit
Delete
Description
API Name
Vendor__c
Enable Reports
Custom
Track Activities
Singular Label
Vendor
Track Field History
33°C
Mostly cloudy
Search
ENG
IN
73%
2:23 PM
10/5/2026


Developer Edition
Search Setup
Setup
Home
Object Manager
SETUP > OBJECT MANAGER
Feedback
Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
API Name
Feedback__c
Custom
Singular Label
Feedback
Plural Label
Feedbacks
Enable Reports
Track Activities
Track Field History
Deployment Status
Deployed
Help Settings
Standard salesforce.com Help Window


Click to go back (Alt+Left arrow), hold to see history
-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01IgK000007k9HB/Detai...
Chat
Developer Edition
Search Setup
Setup
Home
Object Manager
SETUP > OBJECT MANAGER
Event Vendor
Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
API Name
Event_Vendor__c
Custom
Singular Label
Event Vendor
Plural Label
Event Vendors
Enable Reports
Track Activities
Track Field History
Deployment Status
Deployed
Help Settings
Standard salesforce.com Help Window



Activity 1: Create the Event object

1. Click the gear icon → **Setup** → **Object Manager**.
2. Click **Create → Custom Object**.
3. Label: **Event**; Plural Label: **Events**; Record Name: **Event Name**; Data Type: **Text**.
4. Tick **Allow Reports**, **Allow Activities** and **Track Field History**.
5. Deployment Status: **Deployed**; tick **Allow Search**.
6. Tick **Add Notes and Attachments related list to default page layout**.
7. Click **Save**.

Activity 2: Create Client, Venue and Vendor objects

Repeat the same steps with these values and tick **Allow Reports** and **Allow Search** for each.

Label	Plural	Record Name	Data Type
Client	Clients	Client Name	Text
Venue	Venues	Venue Name	Text
Vendor	Vendors	Vendor Name	Text

Activity 3: Create the Feedback object

Label: **Feedback**; Plural: **Feedbacks**; Record Name: **Feedback Number**; Data Type: **Auto Number**; Display Format: **FB-{0000}**; Starting Number: **1**. Tick **Allow Reports** and **Allow Search**, then save.

Activity 4: Create the Event Vendor junction object

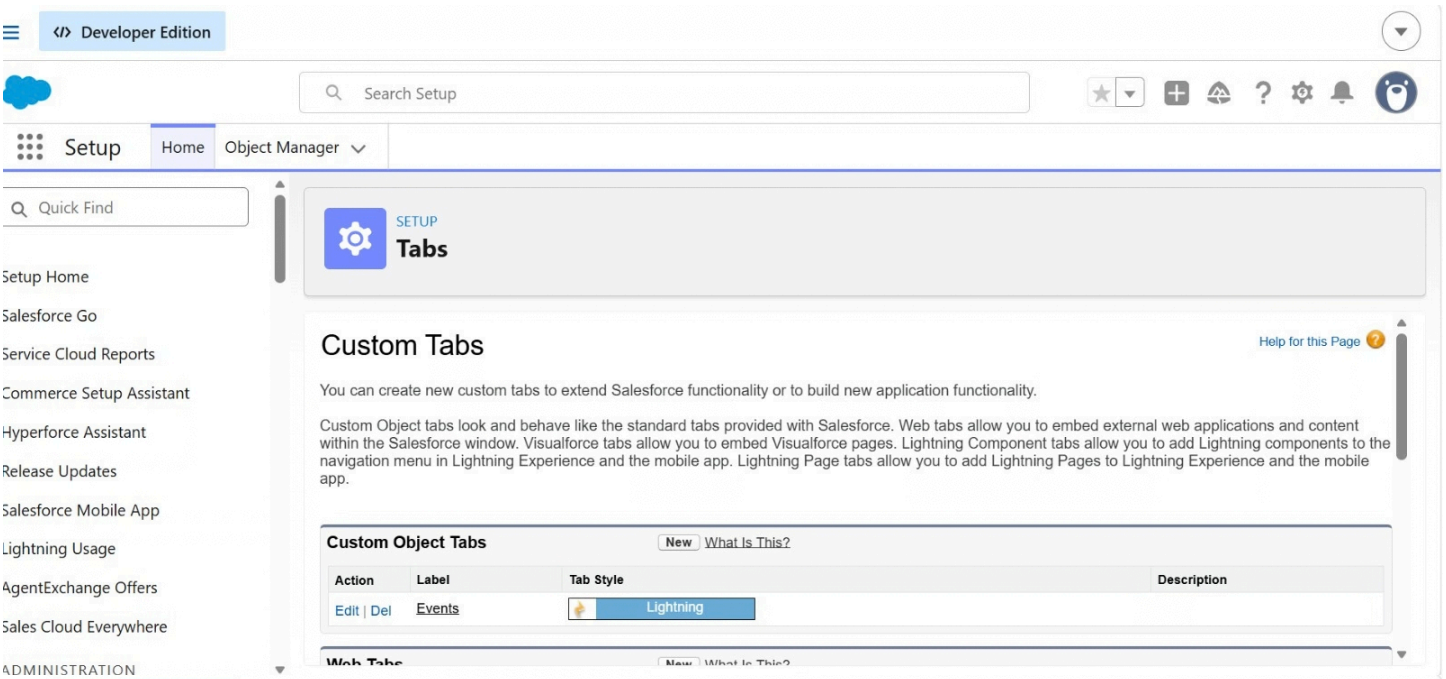
Label: **Event Vendor**; Plural: **Event Vendors**; Record Name: **Event Vendor Number** (Auto Number, format **EV-{0000}**). Master-detail fields are added in Milestone 4.

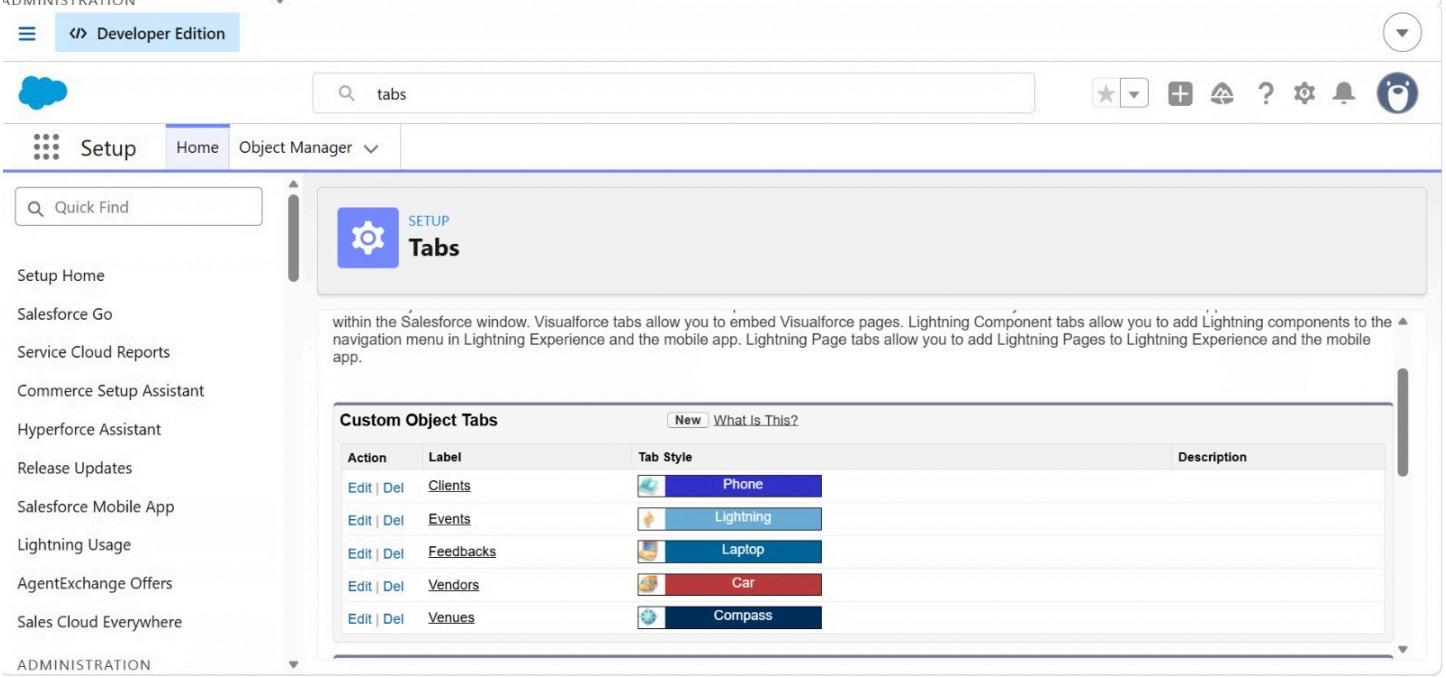
Milestone 3: Tabs

Tabs give users quick access to each object inside an app.

Activity: Create custom tabs for Event, Client, Venue, Vendor and Feedback

1. In Setup, type **Tabs** in Quick Find and select **Tabs**.
2. Under **Custom Object Tabs**, click **New**.
3. Choose the object, pick a tab style (icon) and keep the defaults.
4. Click **Next**, **Next**, **Save**.
5. Repeat for each of the five objects.





Milestone 4: Fields and Relationships

Fields store the data of an object. Standard fields exist by default (Owner, Created By, Last Modified By); custom fields are added by the developer for business needs.

Activity 1: Event fields

Object Manager → **Event** → **Fields & Relationships** → **New**.

Field label	Data type	Values / settings
Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Conference, Exhibition, Concert, Charity Gala, Other
Event Date	Date	Required
Guest Count	Number (6, 0)	Expected number of guests
Event Status	Picklist	Planned, Confirmed, Completed, Cancellation Requested, Cancelled, Cancellation Rejected
Estimated Budget	Formula (Currency)	See formula below

Set **Planned** as the default value of Event Status.

Formula for Estimated Budget

```
CASE( TEXT(Event_Type__c),  "Wedding", 80000,  "Corporate", 50000,  "Birthday", 15000,  "Anniversary", 25000,  "Conference", 60000,  "Exhibition", 70000,  "Concert", 90000,  "Charity Gala", 45000,  20000 )+ ( BLANKVALUE(Guest_Count__c, 0) * 300 )
```

The formula gives a base cost for the event type and adds ₹300 per guest. Use **Check Syntax** before saving.

Activity 2: Client fields

Field label	Data type	Values / settings
Email	Email	–
Phone	Phone	–
Address	Text Area	–
Country	Picklist	India, United States, United Kingdom, United Arab Emirates, Singapore, Australia, Sri Lanka, Malaysia
City	Text (50)	–

Activity 3: Vendor fields

Field label	Data type	Values / settings
Service Type	Picklist (Multi-Select)	Catering, Decoration, Photography, Videography, Sound & Lighting, Stage Setup, Makeup, Live Music / DJ, Transport, Anchoring
Email	Email	–
Phone	Phone	–
Status	Picklist	Active, Booked, Inactive

Activity 4: Venue fields

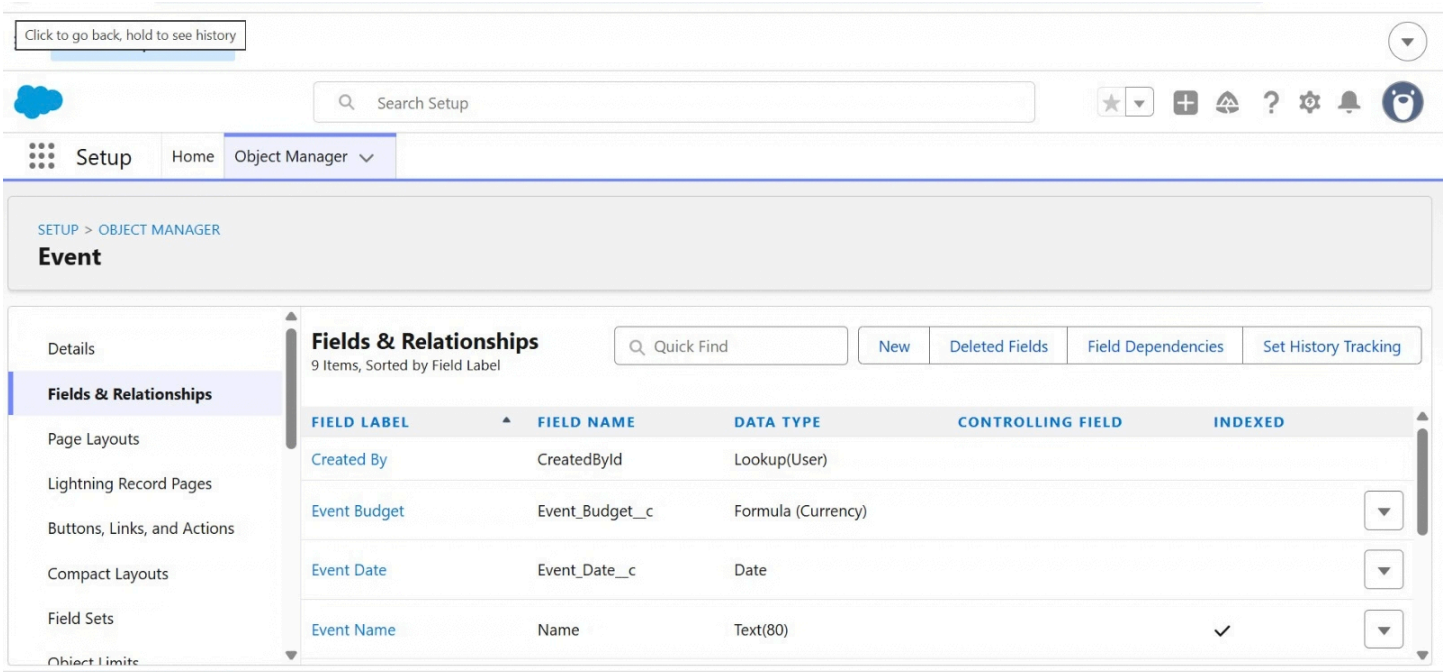
Field label	Data type	Values / settings
Address	Text Area	–
Map Location	URL	Link to the map pin
Seating Capacity	Number (6, 0)	Maximum guests
Availability Status	Picklist	Open, Reserved, Under Maintenance

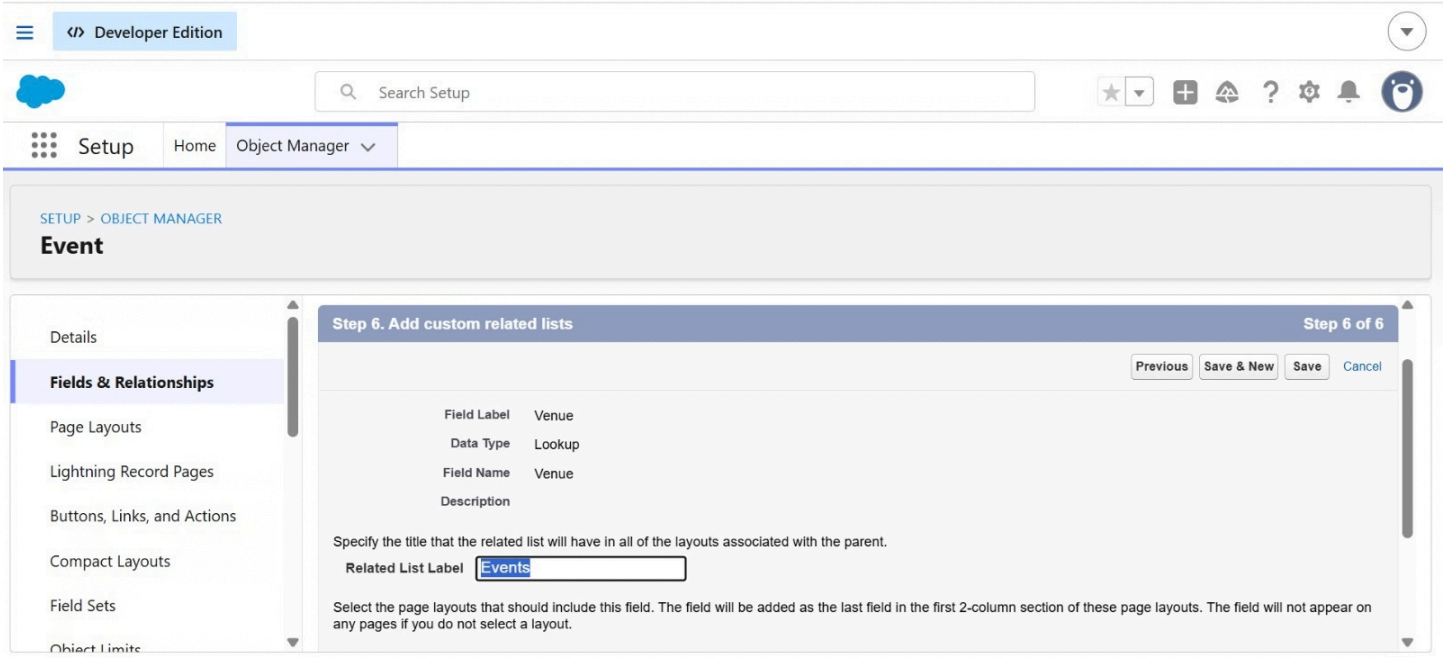
Activity 5: Feedback and Event Vendor fields

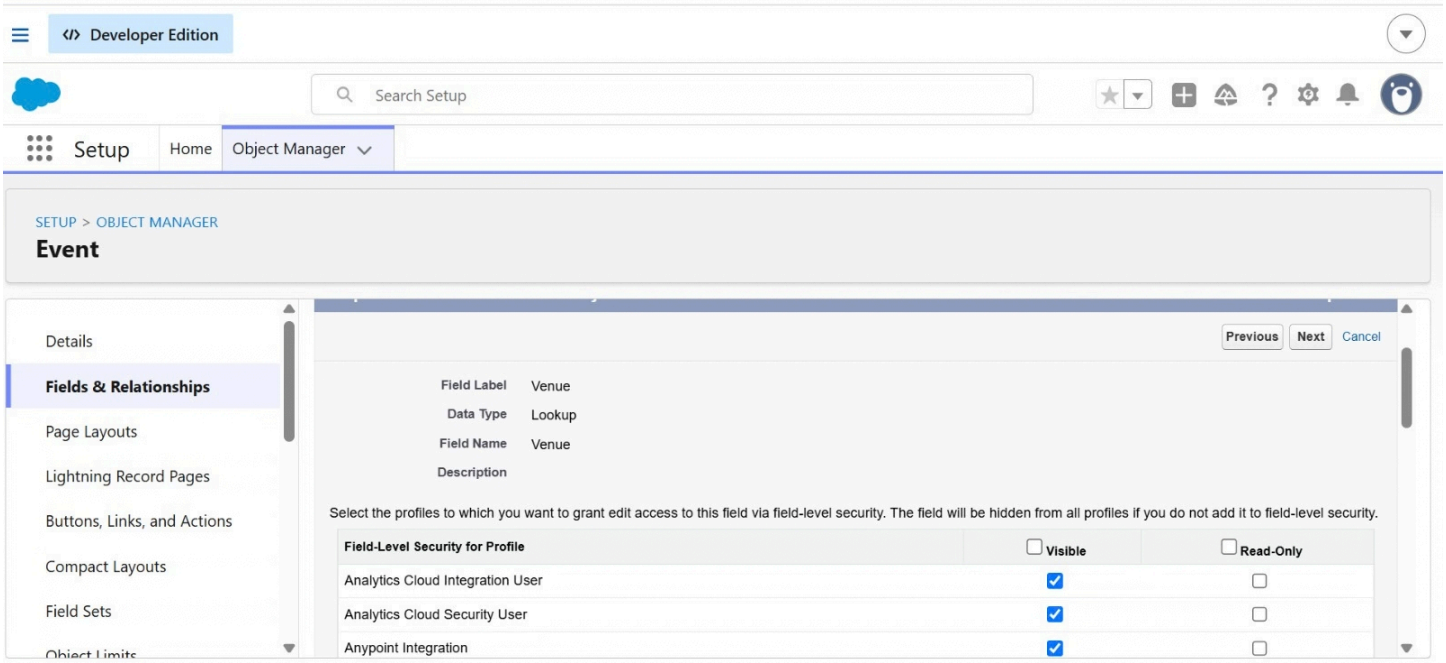
Object	Field label	Data type	Values / settings
Feedback	Rating	Picklist	1, 2, 3, 4, 5
Feedback	Comments	Long Text Area	5000 characters
Event Vendor	Agreed Fee	Currency	–
Event Vendor	Service Notes	Text (255)	–

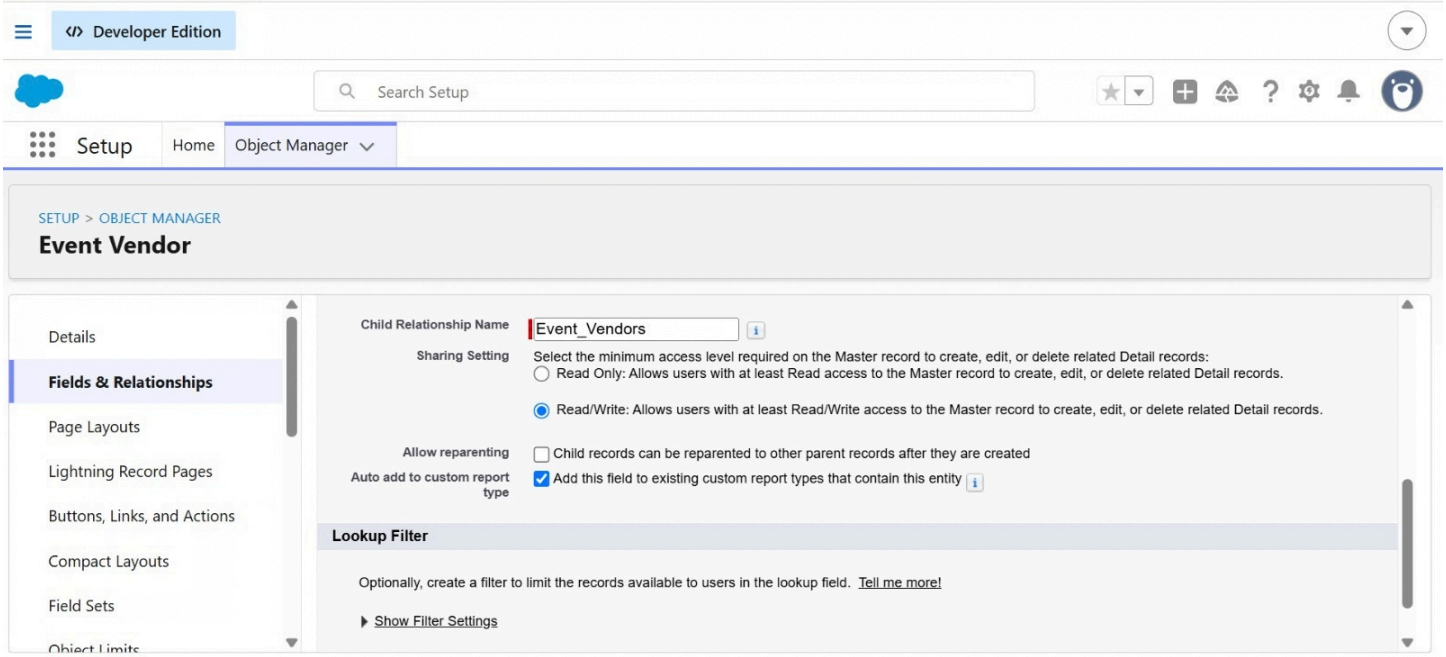
Activity 6: Relationships

On object	Field label	Type	Related to
Event	Client	Lookup	Client
Event	Venue	Lookup	Venue
Event Vendor	Event	Master-Detail	Event
Event Vendor	Vendor	Master-Detail	Vendor
Feedback	Event	Lookup	Event
Feedback	Client	Lookup	Client









Steps for each: Object Manager → select the object → **Fields & Relationships** → **New** → choose Lookup or Master-Detail → select the related object → give the field label → **Next, Next, Next, Save**.

Salesforce does not support a direct many-to-many relationship, so the Event Vendor junction object with two master-detail fields connects many events to many vendors.

Activity 7: Lookup filter on Feedback → Event

Scenario: when a user records feedback for the client Priya, the Event lookup should list only Priya's events.

1. Object Manager → **Feedback** → Fields & Relationships → open the **Event** field → **Edit**.
2. In **Lookup Filter**, choose **Show Filter Settings**.
3. Criteria: **Event: Client equals Feedback: Client** (choose the *Field* option for the right-hand side).
4. Tick **Enable this filter** and **Save**.

Milestone 5: Validation Rules

Validation rules run when a record is saved and show an error if the data breaks a rule.

Rule 1: Valid_Client_Email (on Client)

```
AND(  NOT(ISBLANK(Email__c)),  NOT(REGEX(Email__c, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+[.][a-zA-Z]{2,}$")))
```

Error message: *Please enter a valid email address.* Error location: Field → Email.

Rule 2: No_Past_Event_Date (on Event)

```
AND( ISNEW(), Event_Date__c < TODAY() )
```

Error message: *Event date cannot be in the past.* Error location: Field → Event Date.

Rule 3: Guest_Count_Within_Capacity (on Event)

```
AND(  NOT(ISBLANK(Venue__c)),  NOT(ISBLANK(Guest_Count__c)),  Guest_Count__c >
Venue__r.Seating_Capacity__c)
```

Error message: *Guest count is more than the seating capacity of the selected venue.* Error location: Field → Guest Count. Steps for every rule: Object Manager → object → **Validation Rules** → **New** → enter name → tick **Active** → paste formula → enter message and location → **Save**.

Milestone 6: Approval Process

Note: complete Milestones 14–16 (profiles, roles, users) before activating the Approval Process, because approvers and submitters must exist.

Activity 1: Create email templates (Classic Email Templates)

Setup → **Classic Email Templates** → **New Template** → type **Text** → folder *Unfiled Public Classic Email Templates*.

Template name	Subject	Body
Cancellation Request Notice	Cancellation requested: {!Event__c.Name}	Hello, a cancellation request has been submitted for the event {!Event__c.Name} scheduled on {!Event__c.Event_Date__c}. Please review it in EventForce. Regards, EventForce Team
Cancellation Approved Notice	Approved: cancellation of {!Event__c.Name}	Hello {!Event__c.OwnerFullName}, your request to cancel {!Event__c.Name} (date: {!Event__c.Event_Date__c}) has been approved. The event status is now Cancelled and the venue has been released. Regards, EventForce Team
Cancellation Rejected Notice	Rejected: cancellation of {!Event__c.Name}	Hello {!Event__c.OwnerFullName}, your request to cancel {!Event__c.Name} (date: {!Event__c.Event_Date__c}) has been rejected. The event status is now Cancellation Rejected. Regards, EventForce Team

Activity 2: Create the approval process

1. Setup → **Approval Processes** → object **Event** → **Create New Approval Process → Use Standard Setup Wizard**.
2. Name: **Event Cancellation Approval** (unique name *Event_Cancellation_Approval*).
3. Entry criteria: **Event Status equals Cancellation Requested**.
4. Approver: **Automatically assign to approver(s)** → select the *Event Admin* user.
5. Approval assignment email template: *Cancellation Request Notice*.
6. Fields shown on the approval page: Event Name, Event Type, Event Date, Client, Venue, Guest Count, Estimated Budget, Event Status, Owner.
7. Initial submitters: **Event Owner**.
8. Click **Save**.

Activity 3: Add actions

Stage	Action type	Details
Initial submission	Email Alert	Template: Cancellation Request Notice; recipient: Role – Event Coordinator
Final approval	Field Update	Event Status = **Cancelled**
Final approval	Email Alert	Template: Cancellation Approved Notice; recipient: Event Owner
Final rejection	Field Update	Event Status = **Cancellation Rejected**
Final rejection	Email Alert	Template: Cancellation Rejected Notice; recipient: Event Owner

Activity 4: Add the approval step and activate

1. Click **New Approval Step** → name **Admin Review** → criteria: all records enter this step.
2. Assign the approver as the Event Admin user.
3. On the process detail page click **Activate**.

To use it: a coordinator sets the status to *Cancellation Requested* and clicks **Submit for Approval**.

Milestone 7: Flow – Two-Day Reminder to Client

Scenario: when an event becomes Confirmed and the client has an email address, EventForce emails the client two days before the event date.

Activity 1: Build the flow

1. Setup → **Flows** → **New Flow** → **Record-Triggered Flow** → **Create**.
2. Object: **Event**. Trigger: **A record is created or updated**.
3. Condition: **Event Status equals Confirmed**. When to run: only when the record is updated to meet the condition requirements. Optimise for **Actions and Related Records**.
4. Click the Start element → **Add Scheduled Path**. Label: **Two Days Before**. Time source: **Event Date**. Offset: **2 Days Before**.
5. Create two **Text Template** resources:
   - **ReminderSubject:** `Reminder: {!$Record.Name} is in 2 days`
   - **ReminderBody:**

```
Hello {!$Record.Client__r.Name},This is a friendly reminder that your event "{!$Record.Name}" is on {!$Record.Event_Date__c}.Event type: {!$Record.Event_Type__c}Venue: {!$Record.Venue__r.Name}We look forward to hosting you.- Team EventForce
```

1. Under the scheduled path, click **+** → **Action** → **Send Email**. Label: **Send Client Reminder**.
2. Recipient Address List: `{!$Record.Client__r.Email__c}`. Subject: ReminderSubject. Body: ReminderBody.
3. **Save** with the label **Client Reminder - 2 Days Before** and click **Activate**.

Tip: in Setup → **Deliverability**, set Access Level to **All email** so test emails are delivered.

Milestone 8: Apex Class – Venue Availability Handler

Scenario: a Confirmed event reserves its venue; a Cancelled or Completed event releases it.

Create the class

Gear icon → **Developer Console** → **File → New → Apex Class** → name **VenueAvailabilityHandler**.

```
public with sharing class VenueAvailabilityHandler {    public static void syncVenueStatus(List<Event__c> events) {        Map<Id, String> newStatusByVenue = new Map<Id, String>();        for (Event__c ev : events) {            if (ev.Venue__c == null) { continue;            }            if (ev.Event_Status__c == 'Confirmed') { newStatusByVenue.put(ev.Venue__c, 'Reserved');            } else if (ev.Event_Status__c == 'Cancelled'                    || ev.Event_Status__c == 'Completed') { newStatusByVenue.put(ev.Venue__c, 'Open');            }        }        if (newStatusByVenue.isEmpty()) {            return;        }        List<Venue__c> venuesToUpdate =
```

```
new List<Venue__c>();        for (Id venueId : newStatusByVenue.keySet()) {
venuesToUpdate.add(new Venue__c(                Id = venueId,
Availability_Status__c = newStatusByVenue.get(venueId)            ));        }        update
venuesToUpdate;    }}
```

Create the trigger

File → New → Apex Trigger, name **EventVenueSyncTrigger**, sObject **Event__c**.

```
trigger EventVenueSyncTrigger on Event__c (after insert, after update) {
VenueAvailabilityHandler.syncVenueStatus(Trigger.new);}
```

Save both with **Ctrl + S**.

Milestone 9: Apex Trigger – Prevent Double Booking

Scenario: two active events must not use the same venue on the same date.

File → New → Apex Trigger, name **BlockDoubleBooking**, sObject **Event__c**.

```
trigger BlockDoubleBooking on Event__c (before insert, before update) {    Set<Id> venueIds = new
Set<Id>();    Set<Date> eventDates = new Set<Date>();    for (Event__c ev : Trigger.new) {
if (ev.Venue__c != null && ev.Event_Date__c != null) {            venueIds.add(ev.Venue__c);
eventDates.add(ev.Event_Date__c);        }    }    if (venueIds.isEmpty()) {        return;    }
// key = VenueId + '|' + Date  ->  Id of the event that already holds it    Map<String, Id>
bookedSlots = new Map<String, Id>();    for (Event__c existing : [        SELECT Id, Venue__c,
Event_Date__c        FROM Event__c        WHERE Venue__c IN :venueIds        AND Event_Date__c IN
:eventDates        AND Event_Status__c != 'Cancelled'    ]) {
bookedSlots.put(existing.Venue__c + '|' + String.valueOf(existing.Event_Date__c), existing.Id);
}    for (Event__c ev : Trigger.new) {        if (ev.Venue__c == null || ev.Event_Date__c == null
|| ev.Event_Status__c == 'Cancelled') {            continue;        }        String slot =
ev.Venue__c + '|' + String.valueOf(ev.Event_Date__c);        if (bookedSlots.containsKey(slot) &&
bookedSlots.get(slot) != ev.Id) {            ev.addError('This venue is already booked for the
selected date.');        } else {            bookedSlots.put(slot, ev.Id);        }    }}
```

The trigger uses one query for all records (bulk-safe), ignores cancelled events and also catches two clashing events inserted in the same batch.

Test class (needed for good practice and deployment)

```
@isTestprivate class EventVenueTests {    @isTest    static void secondEventOnSameDateIsBlocked()
{        Venue__c v = new Venue__c(Name = 'Grand Hall', Seating_Capacity__c = 500,
Availability_Status__c = 'Open');        insert v;        Date d = Date.today().addDays(10);
insert new Event__c(Name = 'Event A', Event_Date__c = d, Event_Type__c = 'Corporate',
Event_Status__c = 'Planned', Venue__c = v.Id);        Database.SaveResult result =
Database.insert(            new Event__c(Name = 'Event B', Event_Date__c = d, Event_Type__c =
'Wedding',                        Event_Status__c = 'Planned', Venue__c = v.Id), false);
System.assert(!result.isSuccess(), 'Double booking should be blocked');    }    @isTest    static
void confirmedEventReservesVenue() {        Venue__c v = new Venue__c(Name = 'Sky Lawn',
Seating_Capacity__c = 300,                                    Availability_Status__c = 'Open');
insert v;        insert new Event__c(Name = 'Event C', Event_Date__c = Date.today().addDays(5),
Event_Type__c = 'Birthday', Event_Status__c = 'Confirmed',                                Venue__c =
v.Id);        Venue__c updated = [SELECT Availability_Status__c FROM Venue__c WHERE Id = :v.Id];
System.assertEquals('Reserved', updated.Availability_Status__c);    }}
```

Milestone 10: Batch Apex and Scheduled Apex

Scenario: every night, confirmed events whose date has passed are marked Completed.

Step 1: Batch class – File → New → Apex Class → **BatchMarkEventsCompleted**

```
global class BatchMarkEventsCompleted implements Database.Batchable<SObject> {    global
Database.QueryLocator start(Database.BatchableContext bc) {        return
Database.getQueryLocator(            'SELECT Id, Event_Status__c FROM Event__c ' +
'WHERE Event_Date__c < TODAY AND Event_Status__c = \'Confirmed\''        );    }    global void
execute(Database.BatchableContext bc, List<Event__c> scope) {        for (Event__c ev : scope) {
ev.Event_Status__c = 'Completed';        }        update scope;    }    global void
finish(Database.BatchableContext bc) {        System.debug('Past confirmed events have been marked
Completed.');    }}
```

Step 2: Schedulable class – **EventCompletionScheduler**

```
global class EventCompletionScheduler implements Schedulable {    global void
execute(SchedulableContext sc) {        Database.executeBatch(new BatchMarkEventsCompleted(),
200);    }}
```

Step 3: Schedule the job

1. Setup → **Apex Classes** → **Schedule Apex**.
2. Job Name: **Nightly Event Completion**. Apex Class: **EventCompletionScheduler**.
3. Frequency: **Weekly**; Days: select **all seven days**.
4. Preferred Start Time: **11:00 PM**; set a start and end date.
5. Click **Save**. The job now runs every night.

Because the batch only picks *Confirmed* events, cancelled and rejected events are never changed by mistake.

Phase 3: UI/UX Development & Customization

This phase gives EventForce a clean working area and visual insight. A Lightning App groups every tab in one place, reports turn records into summaries, and a dashboard shows the key numbers on a single screen.

Milestone 11: The Lightning App

An app in Salesforce is a set of tabs that work together, with its own name, logo and navigation bar.

Activity: Create the EventForce app

1. Setup → **App Manager** → **New Lightning App**.
2. App Name: **EventForce**; add a logo image if available; click **Next**.
3. App Options and Utility Items: keep the defaults; click **Next**.
4. Navigation Items: move **Events, Clients, Venues, Vendors, Feedbacks, Reports** and **Dashboards** to *Selected Items*; click **Next**.
5. User Profiles: select **System Administrator**, **Event Admin**, **Event Coordinator** and **Vendor Manager** (add them after Milestone 14 if they do not appear yet); click **Save & Finish**.
6. Open the **App Launcher**, search *EventForce* and open the app to check the navigation bar.

Milestone 12: Reports

Before creating reports, add at least 10 sample records to every object (see Milestone 19).

Activity 1: Create a report folder

1. App Launcher → **Reports** → **New Folder**.
2. Folder Label: **EventForce Reports**; click **Save**.

Activity 2: Share the folder

1. In the *All Folders* list, open the drop-down beside *EventForce Reports* → **Share**.
2. Share with **Roles** → *Event Coordinator* and *Vendor Manager* with **Viewer** access.
3. Click **Share**, then **Done**.

Activity 3: Create the reports

Report	Report type	Columns	Filters	Grouping / summary	Chart
Upcoming Events by Month	Events	Event Name, Event Date, Event Type, Client, Venue, Guest Count, Estimated Budget	Event Date ≥ TODAY; Event Status ≠ Cancelled	Group by Event Date (Calendar Month); Record Count and Sum of Estimated Budget	Column chart
Events by Status	Events	Event Name, Event Date, Event Type	All events	Group by Event Status; Record Count	Donut chart
Feedback by Rating	Feedbacks	Feedback Number, Event, Client, Comments	All records	Group by Rating; Record Count	Bar chart

Steps for each report: **New Report** → choose the report type → **Start Report** → add columns → add filters → group rows → **Add Chart** → **Save** into the *EventForce Reports* folder → **Run**.

Milestone 13: Dashboards

Activity: Create the EventForce Operations Dashboard

1. In the EventForce app, open **Dashboards** → **New Dashboard**.
2. Name: **EventForce Operations Dashboard**; folder: *EventForce Reports*; click **Create**.
3. Click **+ Widget** and add:

Widget	Source report	Chart	Title
1	Upcoming Events by Month	Column	Upcoming Events and Budget
2	Events by Status	Donut	Event Status Overview
3	Feedback by Rating	Bar	Client Satisfaction

1. Click **Save** and then **Done**. Use **Refresh** to see the latest data.

Phase 4: Data Migration, Testing & Security

Before the system is used, sample data was loaded with the Data Import Wizard, users were created with suitable access, and every feature was tested. Organization-Wide Defaults were kept restrictive for events, and sharing rules were used to open access only where needed.

Milestone 14: Profiles

A profile is a collection of settings and permissions that controls what a user can do: object and field access, apps, tabs, login hours and more. A user has exactly one profile.

Activity 1: Event Admin profile

Setup → **Profiles** → **New Profile** → Existing Profile: **System Administrator** → Name: **Event Admin** → **Save**. Cloning the System Administrator gives full access (Modify All Data, Customize Application), which suits the person who owns the application.

Activity 2: Other profiles

For each profile below click **New Profile**, clone **Standard Platform User**, enter the name, **Save**, then **Edit** and set:

- Custom App Settings: default app = **EventForce**.
- Object permissions as per the table.
- Session Settings: session times out after **2 hours** of inactivity.
- Password policies: minimum length **8**; passwords expire in **90 days**.

Object	Event Coordinator	Vendor Manager	Client User
Event	Create, Read, Edit, Delete	Read	Read (own events only)
Client	Create, Read, Edit	Read	No access
Venue	Read	Read	No access
Vendor	Read	Create, Read, Edit, Delete	No access
Event Vendor	Read	Create, Read, Edit, Delete	No access
Feedback	Create, Read	Read	Create, Read

Milestone 15: Roles

Roles decide record visibility through the hierarchy: users higher in the hierarchy can see the records of users below them.

1. Setup → **Roles** → **Set Up Roles** → **Expand All**.
2. Under **CEO**, click **Add Role** → Label **Event Admin** → **Save**.
3. Under *Event Admin* add three roles one by one: **Event Coordinator**, **Vendor Manager** and **Client**.

```
CEO
└─ Event Admin
   ├─ Event Coordinator
   ├─ Vendor Manager
   └─ Client
```

[Screenshot: role hierarchy]

Milestone 16: Users

A user is anyone who logs in to Salesforce. Each user has a profile (what they can do) and a role (which records they can see).

Name (example)	Role	Profile	License
Arun Kumar	Event Admin	Event Admin	Salesforce
Divya Ramesh	Event Coordinator	Event Coordinator	Salesforce Platform
Karthik Selvam	Vendor Manager	Vendor Manager	Salesforce Platform
Priya Nair	Client	Client User	Salesforce Platform

Steps: Setup → **Users** → **New User** → enter first name, last name, alias, email, a unique username (email format) and nickname → select role, license and profile → **Save**. Repeat for all four users.
Note: a Developer Edition org has a very small number of full Salesforce licenses; use *Salesforce Platform* for the other users, or deactivate an unused user to free a license.

Milestone 17: Permission Set

A permission set adds extra access to selected users without changing their profile.

1. Setup → **Permission Sets** → **New**. Label: **Feedback Analyst**; **Save**.
2. Open **Object Settings** → **Feedbacks** → **Edit**.
3. Enable **Read, Create, Edit** and **View All**; click **Save**.
4. Click **Manage Assignments** → **Add Assignment** → select the Event Coordinator user → **Next** → **Assign** → **Done**.

This lets the coordinator review and correct all feedback records without giving the same power to every Event Coordinator profile user.

Milestone 18: Sharing Settings

Organization-Wide Defaults (OWD) set the baseline record access. Other tools (role hierarchy, sharing rules, manual sharing, Apex sharing) can only *open* access further.

OWD option	Meaning
Private	Only the owner and users above in the hierarchy can see the record
Public Read Only	Everyone can see; only owner and above can edit
Public Read/Write	Everyone can see and edit
Controlled by Parent	Access follows the parent record

Activity 1: Set OWD for Event

Setup → **Sharing Settings** → **Edit** → select **Event** → set Default Internal Access to **Private** (and External Access to **Private**) → **Save**.

Activity 2: Create a sharing rule

1. In Sharing Settings, scroll to **Event Sharing Rules** → **New**.
2. Label: **Share Events With Vendor Managers**.
3. Rule type: **Based on record owner**; owned by members of **Roles: Event Coordinator**.
4. Share with: **Roles: Vendor Manager**.
5. Access level: **Read Only** → **Save**.

Vendor Managers can now see events created by coordinators (to plan vendor work) but cannot change them.

Milestone 19: Data Migration – Data Import Wizard

Sample data was prepared in CSV files and loaded using the Data Import Wizard.

Pre-validation checklist

- Remove duplicate rows and blank required values.
- Check that e-mail addresses are valid and dates use one consistent format.
- Make sure picklist values match exactly (for example *Confirmed*, *Open*, *Charity Gala*).
- Use only future event dates (validation rule) and no two events with the same venue and date (Apex trigger).
- Keep at least 10 records per object.

Import order (parents before children): Clients → Venues → Vendors → Events → Event Vendors → Feedbacks.

Steps for each file

1. Setup → **Data Import Wizard** → **Launch Wizard**.
2. Choose **Custom Objects** → select the object → **Add new records**.
3. Upload the CSV file and click **Next**.
4. Map each CSV column to the matching Salesforce field (for lookups such as Client and Venue, map the name column).
5. Click **Start Import** and review the results page for failed rows.

7. FUNCTIONAL & PERFORMANCE TESTING

Each feature was tested in the org with sample users. Screenshots of the results are attached below each test. *Update the Actual Result and Status columns with your own observations before submission.*

Functional test cases

ID	Feature	Test steps	Expected result	Status
TC-01	Event creation	Create an event with type, date, guest count	Record saved; Estimated Budget is calculated	Pass
TC-02	Email validation	Save a client with the email *abc@*	Error: valid email required	Pass
TC-03	Past-date rule	Create an event dated yesterday	Error: date cannot be in the past	Pass
TC-04	Capacity rule	Enter guest count higher than venue capacity	Error on Guest Count	Pass
TC-05	Junction object	Add two vendors to one event	Two Event Vendor records appear in the related list	Pass
TC-06	Lookup filter	Create feedback for a client	Event lookup shows only that client's events	Pass
TC-07	Double booking	Create a second active event at the same venue and date	Error: venue already booked	Pass
TC-08	Venue reserved	Set event status to Confirmed	Venue becomes Reserved	Pass
TC-09	Reminder flow	Confirm an event dated 2 days ahead	Client receives the reminder email	Pass
TC-10	Approval submit	Set Cancellation Requested and submit	Approval request reaches the Event Admin; alert sent	Pass
TC-11	Approval approved	Admin approves	Status = Cancelled; venue = Open; owner emailed	Pass
TC-12	Approval rejected	Admin rejects	Status = Cancellation Rejected; owner emailed	Pass
TC-13	Batch job	Run the scheduler once from Developer Console for an old Confirmed event	Event becomes Completed	Pass

TC-14	Report and dashboard	Refresh the dashboard	Figures match the report data	Pass

Security tests

User	Test	Expected
Event Coordinator	Delete an event	Allowed
Vendor Manager	Edit an event	Not allowed (read only)
Vendor Manager	View coordinator's events	Allowed through sharing rule
Client User	Open Vendor tab	No access
Client User	Create feedback	Allowed

Performance checks

- 200 sample events were imported in one load; the triggers ran in bulk without hitting governor limits because queries are outside loops.
- The batch job processes records in groups of 200.
- Dashboard and reports loaded within a few seconds on the sample data.

8. ADVANTAGES & DISADVANTAGES

Advantages

- **One source of truth:** events, clients, venues, vendors and feedback are linked in one org.
- **Less manual work:** reminders, venue status and completion run automatically.
- **No double bookings:** Apex checks the venue and date before every save.
- **Controlled cancellation:** every cancellation has an approver and an email trail.
- **Better decisions:** live reports and the dashboard replace manual summaries.
- **Secure access:** profiles, roles, OWD and sharing rules limit what each person sees.
- **Good data quality:** validation rules and a prepared import keep records clean.
- **Easy to grow:** new fields, flows and users can be added without redesign.

Disadvantages

- **Academic environment:** the project runs in a Developer Edition org, not in production.
- **Limited licences and email limits:** Developer Edition restricts users and daily emails.
- **Skilled setup needed:** Flows, Apex and security need Salesforce knowledge to maintain.
- **Deployment effort:** moving to production needs sandboxes, change sets or Salesforce DX.
- **Ongoing monitoring:** scheduled jobs, flows and approvals must be checked regularly.

9. CONCLUSION

EventForce shows how Salesforce can turn a paper-and-spreadsheet event business into a connected, automated system. Clients, events, venues, vendors and feedback now live in one platform where they can be searched, reported on and secured.

The project combined custom objects and relationships, validation rules, a formula field, a record-triggered Flow, an Approval Process, Apex triggers and handler classes, Batch and Scheduled Apex, role-based security, a Lightning App, reports and a dashboard. Together they remove double bookings, send reminders on time, control cancellations and give managers a clear view of operations.

The work was completed in a Developer Edition org for learning purposes, but it follows real implementation practice: planning in sprints, building in stages, migrating validated data, testing each feature and securing access. It can be extended into a production-ready solution.

10. FUTURE SCOPE

- **Production deployment:** move the solution to a production org using sandboxes and change sets or Salesforce DX.
- **Client self-service:** an Experience Cloud portal where clients view events and submit feedback.

- **Payment tracking:** record advances and balances against each event.
- **Smarter vendor management:** vendor ratings, price lists and availability calendars.
- **Multi-channel reminders:** add SMS and WhatsApp alongside email.
- **Calendar integration:** sync events with Google or Outlook calendars.
- **Mobile app:** use the Salesforce mobile app for event-day updates.
- **Advanced analytics:** budget-versus-actual and seasonal demand dashboards.
- **Continuous maintenance:** review flows, Apex jobs, validation rules and sharing settings as the business grows.

11. APPENDIX

Section	Contents
A. Custom objects	Event, Client, Venue, Vendor, Event Vendor, Feedback
B. Declarative automation	Formula field, validation rules, Flow, Approval Process, email templates
C. Apex	VenueAvailabilityHandler, EventVenueSyncTrigger, BlockDoubleBooking, EventVenueTests
D. Asynchronous Apex	BatchMarkEventsCompleted, EventCompletionScheduler
E. Security	Profiles, roles, users, permission set, OWD, sharing rule
F. Data migration	Data Import Wizard with prepared CSV files
G. UI and analytics	Lightning App, reports, dashboard
H. Testing	Functional, security and performance test results

GitHub / Demo video link: **Event.mp4**

Thank You